

Franck Pachot, dbi services

12c SQL Plan Directives

Sometimes the optimizer does a bad choice because of misestimated cardinalities. 11g has brought the ability to recognize those errors by the mean of cardinality feedback. That was reactive optimization. 12c has introduced a new optimizer component to make this feedback more proactive for future queries. Let me show you SQL Plan Directives lifecycle on a simple example.

Missing statistics

You have collected system statistics that are relevant for your hardware. You have dictionary statistics that are accurate. And you have a job that collects object statistics so that they are always fresh. How can you have missing statistics? Let's see an example.

I create the following table:

```
SQL> create table DEMO_TABLE as select mod(rownum,2) a,mod(rownum,2)
b,mod(rownum,2) c,mod(rownum,2) d from dual connect by level <=1000;

Table created.

SQL>
SQL> select * from DEMO_TABLE where rownum<=5;
```

A	B	C	D
1	1	1	1
0	0	0	0
1	1	1	1
0	0	0	0
1	1	1	1

You see the pattern: all columns are either 0 or 1

I have statistics on them:

```
SQL> select column_name,num_distinct from user_tab_col_statistics where
table_name='DEMO_TABLE';

COLUMN_NAM NUM_DISTINCT
-----
A                2
B                2
C                2
D                2
```

And statistics are right: each column has two distinct values.

Then let's run the following query:

```
SQL> alter session set statistics_level=all;

Session altered.

SQL> select count(*) count1 from DEMO_TABLE where a=0 and b=0 and c=0
and d=0;

COUNT1
-----
500
```

I've 1000 rows. Half of them have all 0 and half of them have all 1.

This is a simple query but I want to check if the optimizer has estimated the cardinalities correctly.

And the execution plan:

```
SQL> select * from table(dbms_xplan.display_cursor(null,null,'iostats
last'));

PLAN_TABLE_OUTPUT
-----
SQL_ID      86z3n2y70wavg, child number 0
-----
select count(*) count1 from DEMO_TABLE where a=0 and b=0 and c=0 and d=0

Plan hash value: 1839825454

-----
| Id | Operation          | Name       | Starts | E-Rows | A-Rows |
-----
|  0 | SELECT STATEMENT    |            |       1 |        |       1 |
|  1 | SORT AGGREGATE      |            |       1 |        |       1 |
|*  2 | TABLE ACCESS FULL | DEMO_TABLE |       1 |       63 |      500 |
-----

Predicate Information (identified by operation id):
-----
 2 - filter(("A"=0 AND "B"=0 AND "C"=0 AND "D"=0))
```

Here you see the problem: the query returns 500 rows but the optimizer estimated them to 63. That can be a problem in a larger query where the number of rows returned will determine the join method.

Where the problem comes from?

- We know that half of the rows (so 500 here) have all values equal to 0
- The optimizer only know about each column

Then, the CBO estimates 'A=0' to return 1/2 of the rows, then adding 'B=0' returns 1/2 of the rows etc. So the optimizer estimate the whole combination of 'A=0 and B=0 and C=0 and D=0' to return 1/16th of the rows, which is 63.

Solutions in 11g

What are the solutions for that? In 11g we have two solutions:

- Extended statistics: we calculate statistics on the group of columns (A,B,C,D) so that the optimizer is aware of the correlation
- Dynamic sampling: when dynamic sampling level is at least 4 then complex where clauses like that one will trigger dynamic sampling. And in any sample, the optimizer will see that the predicate returns half of the rows.

12c does not add new solutions for that (except Adaptive Plan that will lower the bad consequence of misestimating) but will bring automation to those two alternatives, in an adaptive way. Let's see Automatic Reoptimization and SQL Plan Directives.

Automatic Reoptimization

If I check the new 'IS_REOPTIMIZABLE' column from V\$SQL, I see that it is flagged to 'Y' for that query:

```
SQL> select sql_id,child_number,is_reoptimizable from v$sql where sql_
id='86z3n2y70wavg';

SQL_ID          CHILD_NUMBER I
-----
86z3n2y70wavg          0 Y
```

During execution, oracle has seen that the actual number of rows (the A-Rows above) is very different from the estimated ones (E-Rows). Then the statement is flagged for re-optimization: the next execution will hard parse again instead of sharing the cursor which is known to have been built on bad estimations.

And I can have more information with the 'report' format from dbms_xplan:

```
SQL> select * from table(dbms_xplan.display_cursor('86z3n2y70wavg',null,'
report'));

Reoptimized plan:
-----
This cursor is marked for automatic reoptimization. The plan that is
expected to be chosen on the next execution is displayed below.

Plan hash value: 1839825454

-----
| Id | Operation          | Name       | Rows |
-----
|  0 | SELECT STATEMENT    |            |     1 |
|  1 | SORT AGGREGATE      |            |     1 |
|*  2 | TABLE ACCESS FULL | DEMO_TABLE |    500 |
-----

Predicate Information (identified by operation id):
-----
 2 - filter("A"=0 AND "B"=0 AND "C"=0 AND "D"=0)
```

I don't reproduce the first part which is the same as the one above, but the 'report' format adds the following:

```
Reoptimized plan:
-----
This cursor is marked for automatic reoptimization. The plan that is
expected to be chosen on the next execution is displayed below.

Plan hash value: 1839825454

-----
| Id | Operation          | Name       | Rows |
-----
|  0 | SELECT STATEMENT    |            |     1 |
|  1 | SORT AGGREGATE      |            |     1 |
|*  2 | TABLE ACCESS FULL | DEMO_TABLE |    500 |
-----

Predicate Information (identified by operation id):
-----
 2 - filter("A"=0 AND "B"=0 AND "C"=0 AND "D"=0)
```

Not only I know that the statement will be reoptimized, but I also have the future plan.

And you see that the estimations (Rows) are now accurate because the optimizer has used the statistics from the previous execution (this comes from cardinality feedback).

SQL Plan directive

So, the optimizer has seen that there were a cardinality misestimate. That was reactive: the previous execution may have been executed with a bad plan. But that knowledge can help future query executions.

This is what SQL Plan Directives are: it stores persistently this information to help future executions.

Persistent means that it is not only in memory, but stored in the dictionary. So in order to see them, we have to flush them:

```
SQL> exec dbms_spd.flush_sql_plan_directive;

PL/SQL procedure successfully completed.
```

And we can see which information is stored in the dictionary:

```
SQL> select directive_id,type,state,reason,created,last_modified,last_used,notes from dba_sql_plan_directives where
directive_id in(select directive_id from dba_sql_plan_dir_objects where owner='DEMO' );
```

DIRECTIVE_ID	TYPE	STATE	REASON
1350980939868665098	DYNAMIC_SAMPLING	USABLE	SINGLE TABLE CARDINALITY MISESTIMATE

```

NOTES
-----
<spd_note>
<internal_state>NEW</internal_state>
<redundant>NO</redundant>
<spd_text>{EC(DEMO.DEMO_TABLE)[A, B, C, D]}</spd_text>
</spd_note>
```

A new directive has been created.

For the moment (12.1) directives are only created for optimizer bad decision for a 'CARDINALITY MISESTIMATE', the type being type 'DYNAMIC_SAMPLING'. The misestimate can concern several kind of operations: single table access, join, group by etc.

The state is 'NEW': the directive has been created but without more information about statistics.

The note gives an idea about the object concerned, which is detailed in the following view.

```
SQL> select object_name,subobject_name,object_type,notes from dba_sql_plan_dir_objects where owner='DEMO' ;
```

OBJECT_NAME	SUBOBJECT_NAME	OBJECT_TYPE	NOTES
DEMO_TABLE	A	COLUMN	
DEMO_TABLE	B	COLUMN	
DEMO_TABLE	C	COLUMN	
DEMO_TABLE	D	COLUMN	
DEMO_TABLE		TABLE	

```

<obj_note>
<equality_predicates_only>YES</equality_predicates_only>
<simple_column_predicates_only>YES</simple_column_predicates_only>
<index_access_by_join_predicates>NO</index_access_by_join_predicates>
<filter_on_joining_object>NO</filter_on_joining_object>
</obj_note>
```

What is very interesting here is that the directive is not assigned to a statement, but only with a combination of tables, columns and kind of predicates.

Here the directive can be used by any statement on the DEMO_TABLE where columns A,B,C,D are involved.

Usable SQL Plan directive

What happens if I run the same query again?

```
SQL> select count(*) count1 from DEMO_TABLE where a=0 and b=0 and c=0 and d=0;

COUNT1
-----
      500

SQL> select * from table(dbms_xplan.display_cursor(null,null,'report iostats last'));

PLAN_TABLE_OUTPUT
-----
SQL_ID      86z3n2y70wavg, child number 1
-----
select count(*) count1 from DEMO_TABLE where a=0 and b=0 and c=0 and d=0

Plan hash value: 1839825454

-----
| Id | Operation          | Name       | Starts | E-Rows | A-Rows |
-----|-----|-----|-----|-----|-----|
|  0 | SELECT STATEMENT    |            |       1 |        |       1 |
|  1 |   SORT AGGREGATE    |            |       1 |        |       1 |
|* 2 |    TABLE ACCESS FULL | DEMO_TABLE |       1 |      500 |      500 |
-----

Predicate Information (identified by operation id):
-----
  2 - filter(("A"=0 AND "B"=0 AND "C"=0 AND "D"=0))

Note
-----
   - statistics feedback used for this statement
```

Thanks to automatic reoptimization the statement has been hard parsed again, now using the cardinality feedback (which is called statistics feedback in 12c) to get accurate estimations.

We have a new child number:

```
SQL> select sql_id,child_number,is_reoptimizable from v$sql where sql_id='86z3n2y70wavg';

SQL_ID      CHILD_NUMBER I
-----
86z3n2y70wavg      0 Y
86z3n2y70wavg      1 N
```

And the internal state of the SQL Plan directive has changed from NEW to MISSING_STATS because the cardinality misestimate has been determined to be caused by missing stats (for correlated columns).

```
SQL> select directive_id,type,state,reason,created,last_modified,last_used,notes from dba_sql_plan_directives where
directive_id in(select directive_id from dba_sql_plan_dir_objects where owner='DEMO' );

DIRECTIVE_ID TYPE      STATE      REASON
-----
NOTES
-----
1350980939868665098 DYNAMIC_SAMPLING USABLE      SINGLE TABLE CARDINALITY MISESTIMATE
<spd_note>
<internal_state>MISSING_STATS</internal_state>
<redundant>NO</redundant>
<spd_text>{EC(DEMO.DEMO_TABLE){A, B, C, D}}</spd_text>
</spd_note>
```

Note that in 12.1.0.2 the state will show USABLE for both 'NEW' and 'MISSING STATS' and we have to check the NOTES column to get the internal state. It has changed from 2.1.0.1 where the state has all the details.

So, by executing the same statement, the SQL Plan Directive was not used (the statement was to be reoptimized anyway) but has been updated to validate its state (missing statistics).

Previously I said that SQL Plan Directive are also used with any statement that uses the same table. Let's change the statement (here changing count by max):

```
SQL> select max(a) from DEMO_TABLE where a=0 and b=0 and c=0 and d=0;
```

MAX(A)	
0	

```
SQL> select * from table(dbms_xplan.display_cursor(null,null,'report iostats last'));

PLAN_TABLE_OUTPUT
-----
SQL_ID  crqtabczzmmdv, child number 0
-----
select max(a) from DEMO_TABLE where a=0 and b=0 and c=0 and d=0

Plan hash value: 1839825454

-----
| Id | Operation          | Name      | Starts | E-Rows | A-Rows |
-----|-----|-----|-----|-----|-----|
| 0  | SELECT STATEMENT    |           |      1 |        |      1 |
| 1  | SORT AGGREGATE      |           |      1 |        |      1 |
|* 2  | TABLE ACCESS FULL  | DEMO_TABLE |      1 |    500 |    500 |
-----

Predicate Information (identified by operation id):
-----
 2 - filter(("A"=0 AND "B"=0 AND "C"=0 AND "D"=0))

Note
-----
- dynamic statistics used: dynamic sampling (level=2)
- 1 Sql Plan Directive used for this statement
```

Then, for the same predicates – but different query- we have good estimations event for the first executions. And the dbms_xplan notes explains that:

- The SQL Plan Directive told the optimizer that some missing statistics can lead to misestimate with those predicates
- Then the optimizer has chosen to gather more stats with Dynamic Sampling

This is the first very nice feature of SQL Plan Directives: the optimizer knows that it will need to do more work in order to have good estimations.

DBMS_STATS

This is good, but not enough. Using Dynamic Sampling is only a short term solution. The durable way to solve the issue is to gather extended statistics for those correlated columns. And this is where SQL Plan Directives are even nicer: dbms_stats will check the directives and will gather the required statistics automatically.

Let's gather the status as usual, without creating any extended statistics ourselves:

```
SQL> exec dbms_stats.gather_table_stats(user,'DEMO_TABLE');

PL/SQL procedure successfully completed.
```

And see what has been gathered:

```
SQL> select extension_name,extension from user_stat_extensions where table_name='DEMO_TABLE';

EXTENSION_ EXTENSION
-----
SYS_STSPJN ("A","B","C","D")
```

We have now statistics on those columns, because dbms_stats knew that they were missing when optimizing some queries.

For the moment, the directive state has not changed. But let's run another query (I've changed max to min):

```
SQL> exec dbms_spd.flush_sql_plan_directive;

PL/SQL procedure successfully completed.

SQL> select directive_id,type,state,reason,created,last_modified,last_used,notes from dba_sql_plan_directives where
directive_id in(select directive_id from dba_sql_plan_dir_objects where owner='DEMO' );

DIRECTIVE_ID TYPE          STATE          REASON
-----
NOTES
-----
1350980939868665098 DYNAMIC_SAMPLING SUPERSEDED SINGLE TABLE CARDINALITY
<spd_note>
<internal_state>HAS_STATS</internal_state>
<redundant>NO</redundant>
<spd_text>{EC(DEMO.DEMO_TABLE)[A, B, C, D]}</spd_text>
</spd_note>
```

The internal state is now 'HAS_STATS', the displayed state being 'SUPERSEDED' which means that the problem has been fixed: we have enough statistics now.

Why did we need another execution to get that state? Because the optimizer has checked that the issue is now fixed permanently. If it were not the case, then the internal state would have been 'PERMANENT', meaning that the misestimate is permanent and the optimizer cannot rely on the gathered extended statistics.

Conclusion

That new component helps to share and store what the optimizer observes at execution time. It is intelligence and memory that are added to the cardinality feedback features. Those observation lead to dynamic sampling as a short term solution and to extended statistics gathering for long term solutions. When an issue is fixed (and the optimizer checks that as well) then the SQL Plan directives are automatically purges after one year. ■

Contact

dbi services

Franck Pachot

E-Mail:

franck.pachot@dbi-services.com